

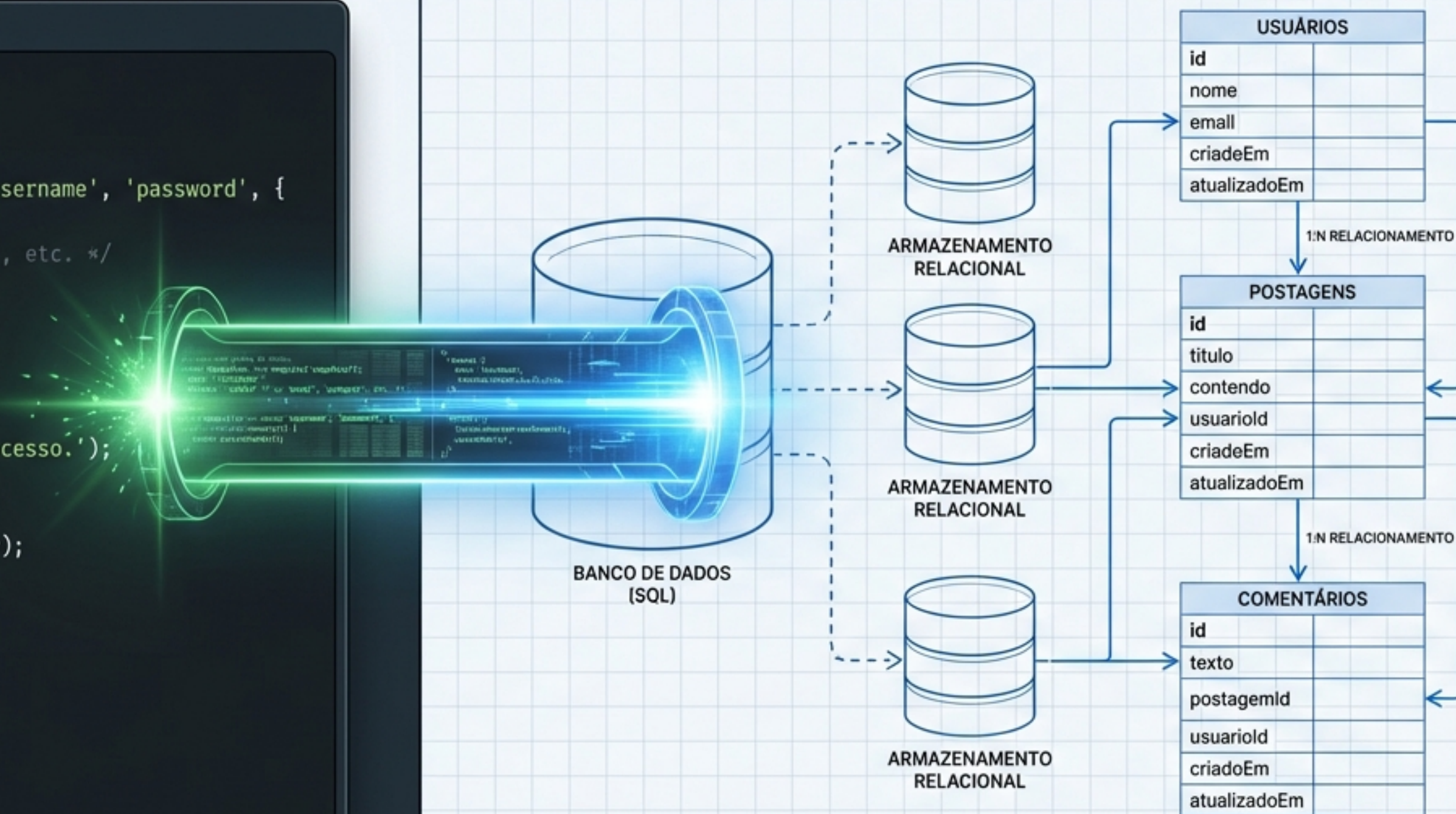
Fundamentos do Sequelize

O Motor de Dados do Node.js

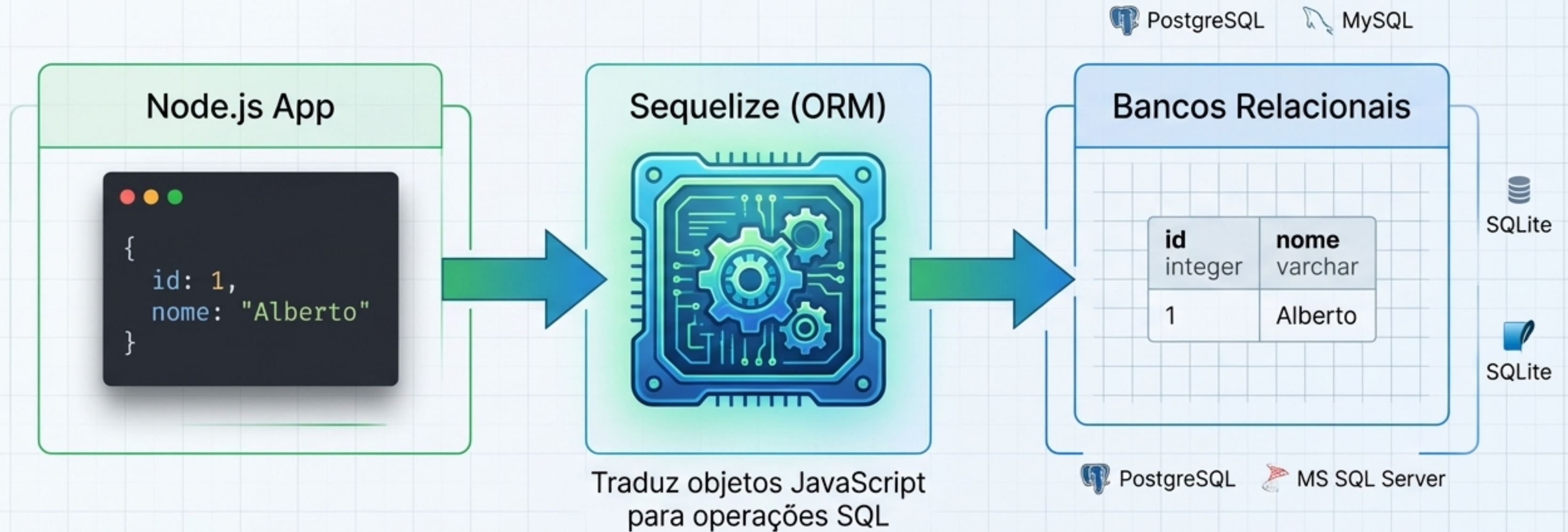
```
// Conexão do Banco de Dados
const { Sequelize } = require('sequelize');

const sequelize = new Sequelize('database', 'username', 'password', {
  host: 'localhost',
  dialect: 'sqlite' /* ou 'mysql', 'postgres', etc. */
});

async function conectar() {
  try {
    await sequelize.authenticate();
    console.log('Conexão estabelecida com sucesso.');
```



O que é o Sequelize?

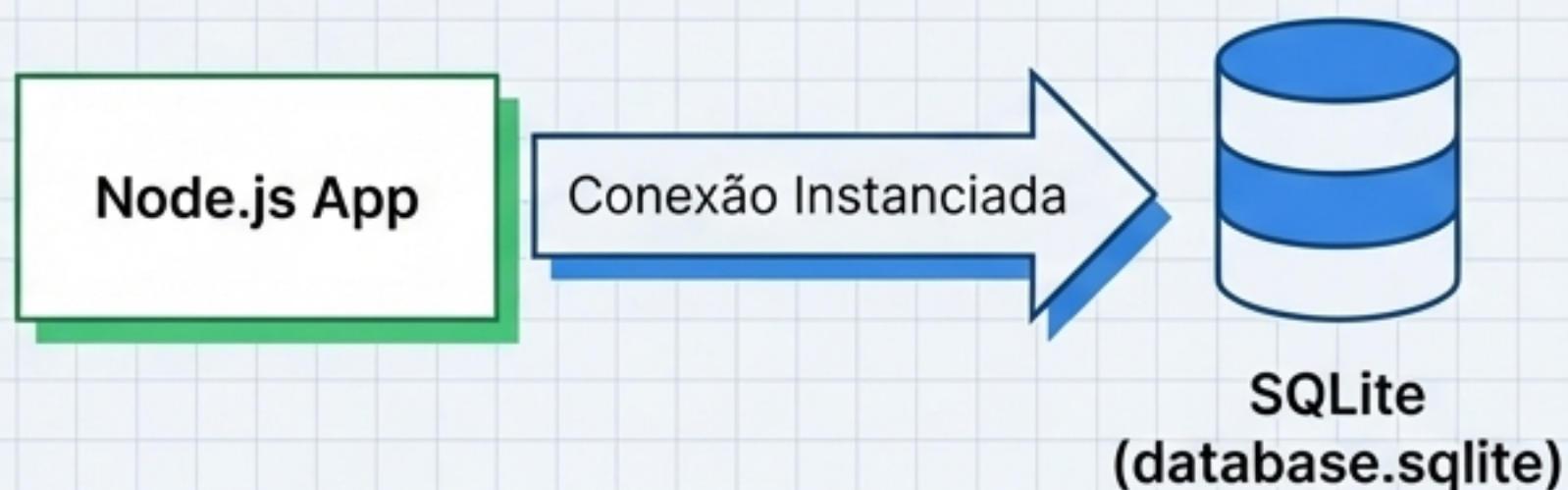


- Representar tabelas como classes JS.
- Abstrair consultas SQL sem escrever strings cruas.
- Gerenciar relacionamentos de forma automatizada.

O Ponto de Partida: Conectando a Aplicação

```
const { Sequelize } = require("sequelize");

const sequelize = new Sequelize({
  dialect: 'sqlite',
  storage: 'database.sqlite'
});
```



O objeto **sequelize** gerado torna-se o motor central para todas as operações subsequentes de banco de dados.

O Blueprint: Definindo Modelos

```
class Cliente extends Model {  
  static init(sequelize) {  
    super.init({ nome: DataTypes.STRING },  
      { sequelize, modelName: 'cliente', tableName: 'clientes' })  
  }  
}  
Cliente.init(sequelize); // CREATE TABLE
```

SQL Executado:
CREATE TABLE clientes

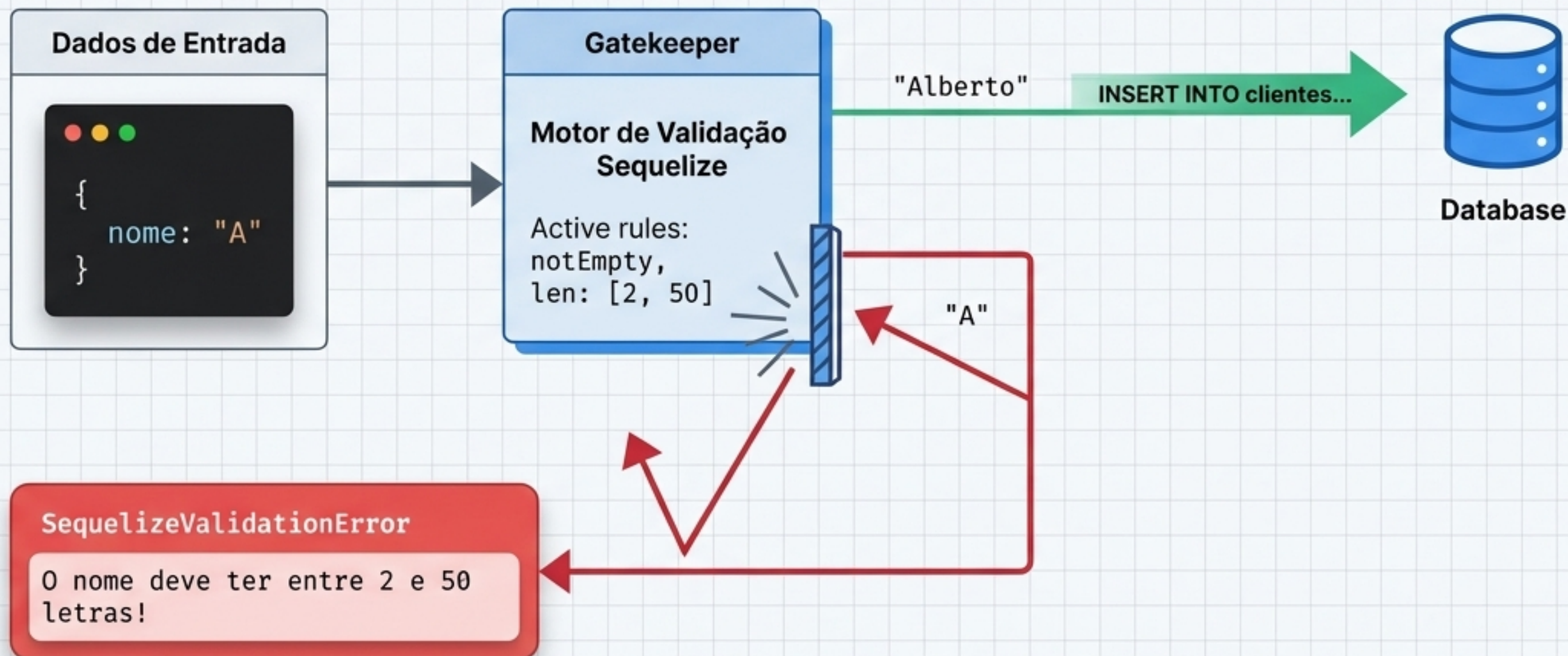
Tabela: clientes	
PK	id (PK)
	nome
	createdAt
	updatedAt

Classe JavaScript

Coluna: nome (String)

O Sequelize injeta automaticamente chaves primárias (id) e timestamps (createdAt, updatedAt).

O Guardião: Validação de Dados em Memória



Regras Comuns

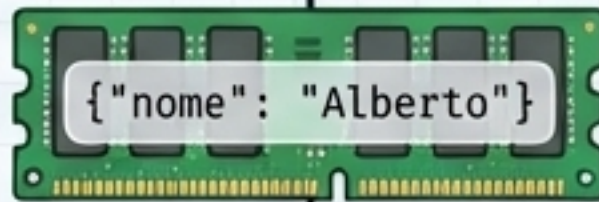
isEmail
isNumeric
isURL
isInt
notNull

Validações ocorrem ANTES do SQL. Se falhar, o banco de dados nem é consultado.

Ciclo de Vida do Dado: Memória vs. Disco

Duas Etapas (Memória → Disco)

```
const c1 = Cliente.build({ nome: "Alberto" });
```



Apenas em memória!
Sem ID.

```
await c1.save();
```

INSERT INTO clientes...



Gera ID e Timestamps
no Disco.

Uma Etapa (Direto ao Disco)

```
const c2 = await Cliente.create({ nome: "Bernardo" });
```

Combina build() + save()
em uma única operação.



Nota sobre Atualizações

O método `save()` é inteligente: se o objeto já tem um `id`, ele gera um `UPDATE` em vez de um `INSERT`.

A Caixa de Ferramentas: Métodos de Consulta

`findAll()`

Retorna um array de instâncias (Todos os registros).

```
SELECT * FROM clientes
```

`findAll({ attributes: ['nome'] })`

Retorna array, mas otimiza a busca limitando as colunas.

```
SELECT nome FROM clientes
```

`findAll({ where: { id: 3 } })`

Retorna array filtrado rigorosamente por cláusulas condicionais.

```
SELECT * FROM clientes WHERE id = 3
```

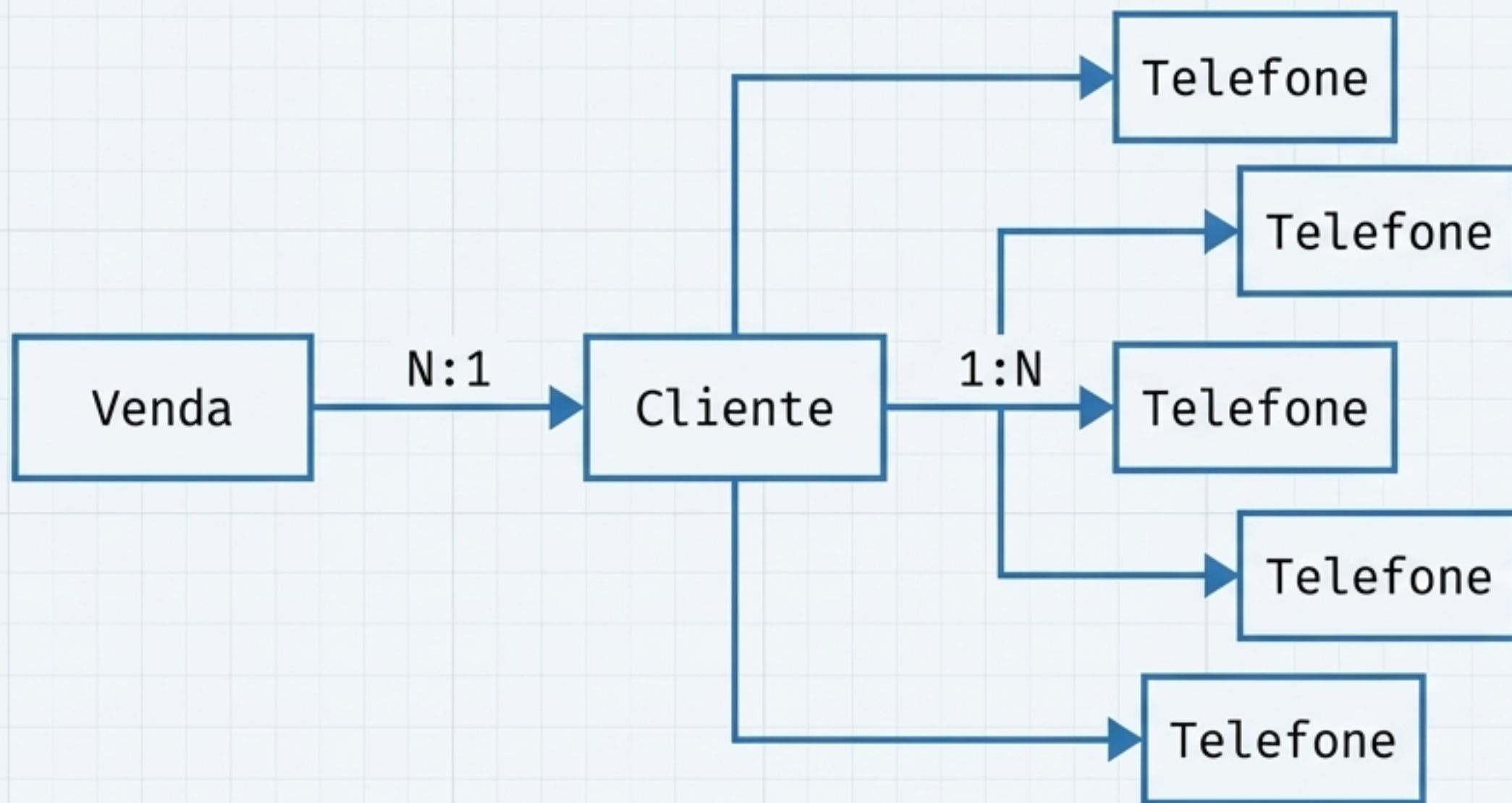
`findByPk(3) / findOne(...)`

Retorna um único objeto (não um array). Altamente otimizado para buscas diretas.

```
SELECT * FROM clientes WHERE id = 3 LIMIT 1
```


A Rede: Introdução às Associações

Bancos relacionais ganham poder nas conexões. O Sequelize mapeia essas relações estruturais para o JavaScript.



Quatro Pilares das Associações

- 1. **hasOne**: 1:1 (Um para um)
- 2. **hasMany**: 1:N (**Um para muitos**) - **Foco Principal**
- 3. **belongsTo**: N:1 (Muitos para um) - **Foco Principal**
- 4. **belongsToMany**: N:N (Muitos para muitos)

O Mapa Relacional: Injetando Chaves Estrangeiras

clientes	
🔑	id
	nome

telefones	
🔑	id
	numero
→	clienteId (FK)

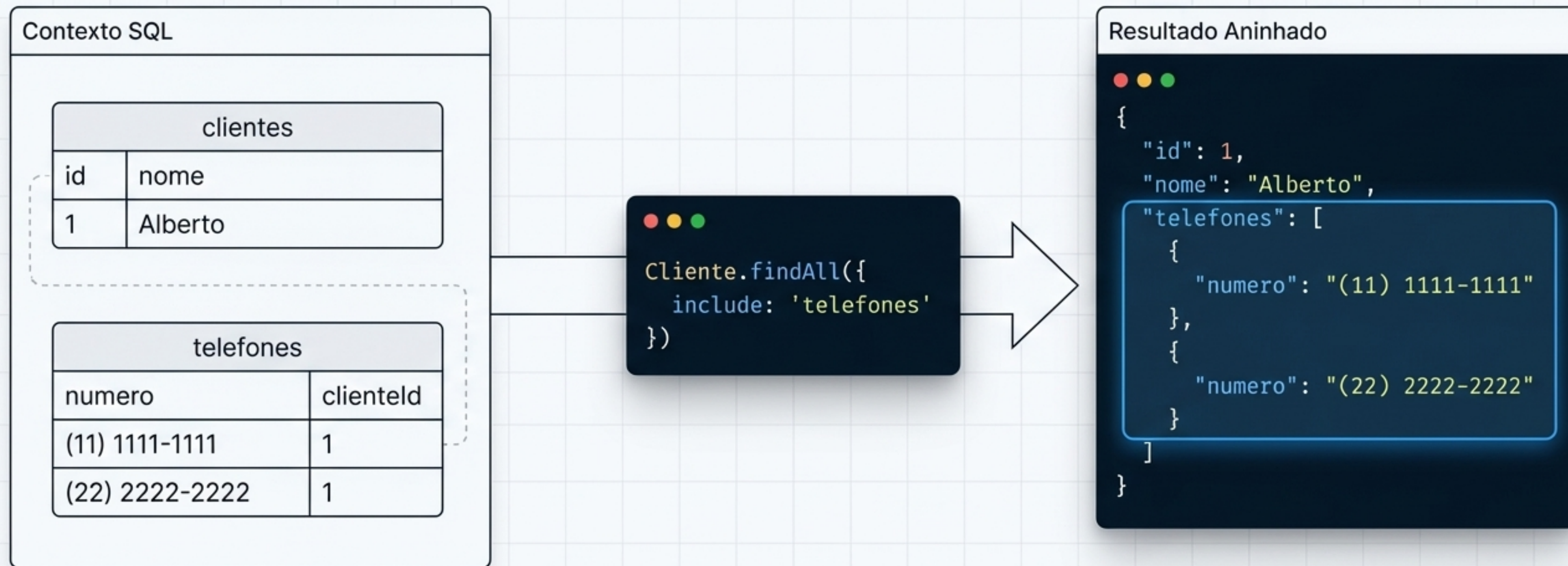
```
Cliente.hasMany(Telefone, {as: 'telefones'});
```

```
Venda.belongsTo(Cliente);
```

vendas	
🔑	id
	valor
→	clienteId (FK)

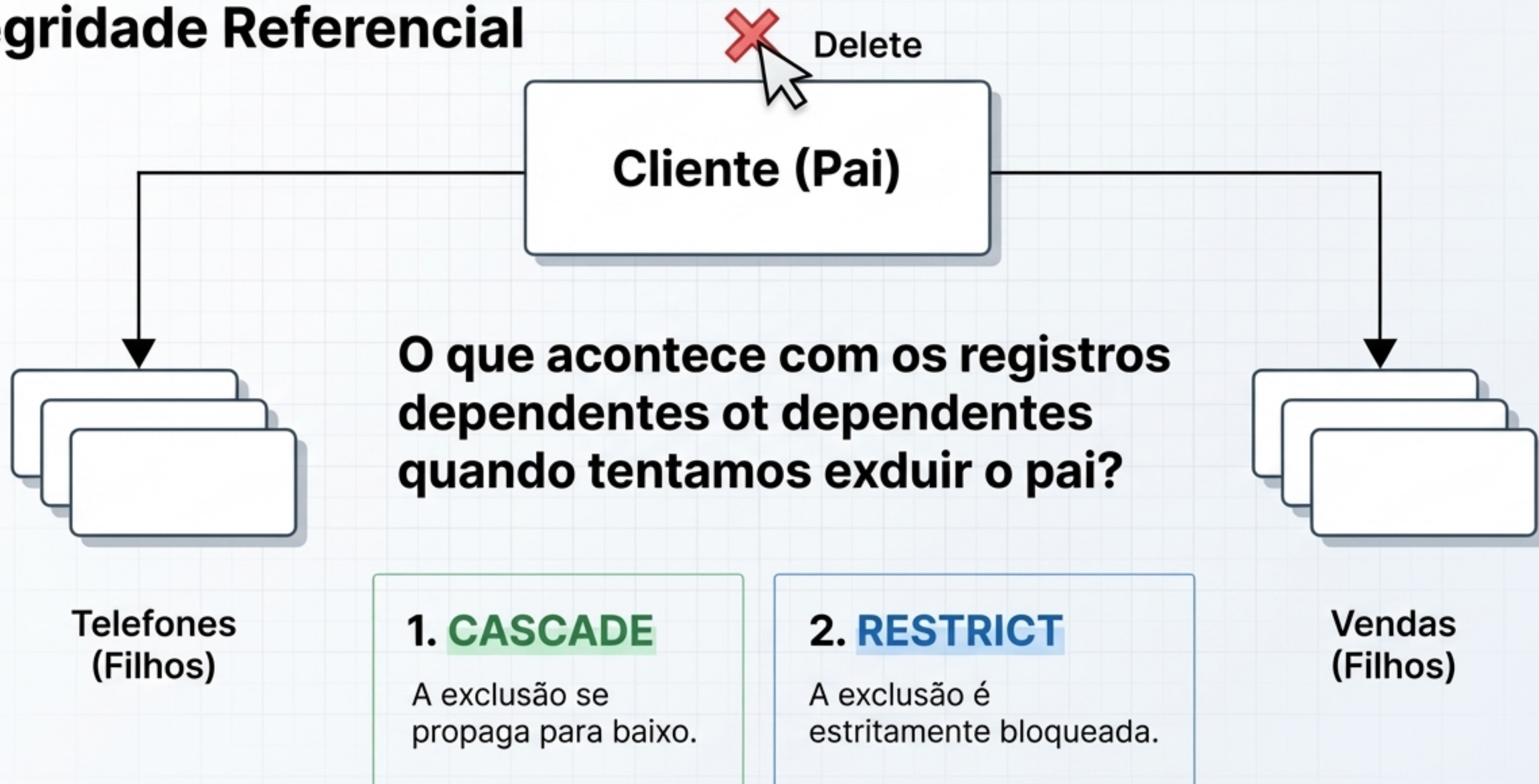
O **Sequelize** cria automaticamente as Chaves Estrangeiras (FK) e injeta métodos mágicos nas classes, como `cliente.addTelefone()` e `venda.setCliente()`.

O Agregador: Consultas com include



O include substitui a necessidade de escrever JOINS manuais, entregando dados perfeitamente aninhados para sua API.

A Rede de Segurança: Integridade Referencial



O Efeito Dominó vs. A Parede

CASCADE (🔄) - O Efeito Dominó



A exclusão do pai propaga em cascata para os filhos.
Útil para dados dependentes (telefones, configurações).

RESTRICT (🚫) - A Parede

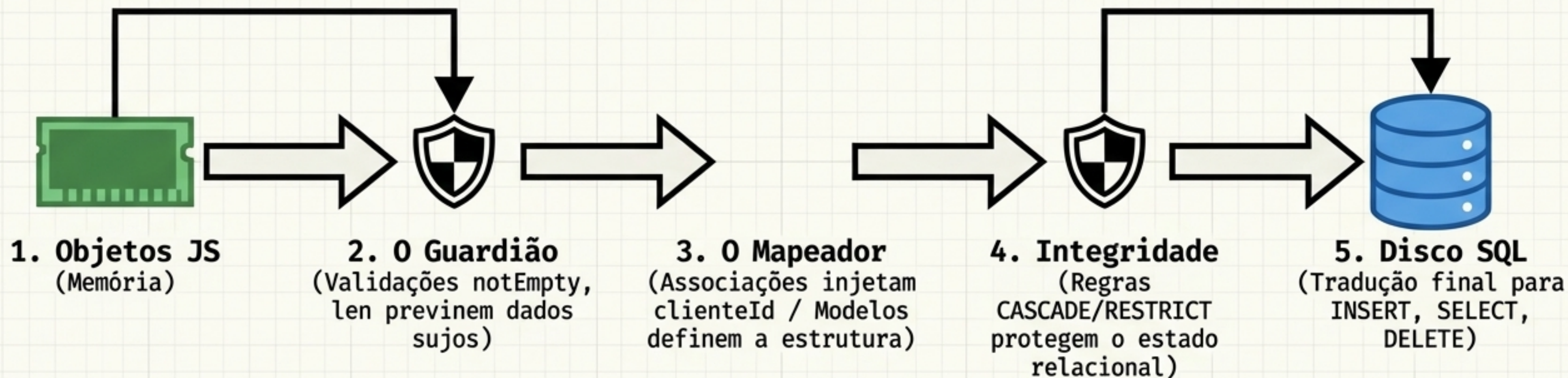


A existência do filho bloqueia a exclusão do pai.
Protege dados críticos (vendas, histórico financeiro).

Matriz de Diagnóstico: Regras de Integridade

Característica	CASCADE	RESTRICT
Ao Excluir Pai	Exclui filhos automaticamente	Bloqueia a exclusão (lança erro)
Ao Atualizar Pai	Atualiza FK dos filhos	Bloqueia a atualização
Protege dados filhos?	Não (dados perdidos junto com o pai)	Sim (filhos são preservados)
Uso Ideal	Telefones, Endereços (não fazem sentido sem o pai)	Vendas, Transações, Históricos (dados intocáveis)
SQL Gerado	ON DELETE CASCADE	ON DELETE RESTRICT
Outras alternativas suportadas: SET NULL (Anula a FK do filho) e NO ACTION (Similar ao Restrict).		

Síntese: A Arquitetura Sequelize em Ação



O Sequelize não é apenas um tradutor de SQL; é um motor completo de validação, relacionamento e proteção de dados.